

Table A.1: Action programs for one negative and one positive example in the BD instance.

Negative Action Program	Positive Action Program
line_normal_0.300-0.500,	line_normal_1.000-0.500,
line_normal_0.424-0.875,	line_normal_0.283-0.875,
line_normal_0.300-0.875,	line_normal_0.200-0.875,
line_normal_0.800-0.167,	line_normal_0.583-0.086,
line_normal_0.800-0.833,	line_normal_0.500-0.664,
line_normal_0.800-0.833,	line_normal_0.500-0.750,
line_normal_0.200-0.500,	line_square_0.200-0.500,
arc_zigzag_0.500_0.625-0.500,	arc_triangle_0.500_0.625-0.500,
line_normal_0.400-0.500,	line_normal_0.400-0.500,
arc_triangle_0.500_0.625-0.500,	arc_square_0.500_0.625-0.500,
line_normal_0.200-0.500,	line_circle_0.200-0.500,
arc_triangle_0.500_0.625-0.500,	arc_normal_0.500_0.625-0.500,
line_triangle_0.400-0.500,	line_triangle_0.400-0.500,
arc_normal_0.500_0.625-0.500	arc_triangle_0.500_0.625-0.500

Negative example.

To draw figure 1, follow these steps:

- Step 1: draw a normal line of 0.300 units. Then, continue straight without turning.
- Step 2: draw a normal line of 0.424 units. After that, turn left by 135.0 degrees.
- Step 3: draw a normal line of 0.300 units. After that, turn left by 135.0 degrees.
- Step 4: draw a normal line of 0.800 units. After that, turn right by 119.9 degrees.
- Step 5: draw a normal line of 0.800 units. After that, turn left by 119.9 degrees.
- Step 6: draw a normal line of 0.800 units. After that, turn left by 119.9 degrees.
- Step 7: draw a normal line of 0.200 units. Then, continue straight without turning.
- Step 8: draw a zigzag arc with a radius of 0.500 and sweeping 90.0 degrees. Then, continue straight without turning.
- Step 9: draw a normal line of 0.400 units. Then, continue straight without turning.
- Step 10: draw a triangle arc with a radius of 0.500 and sweeping 90.0 degrees. Then, continue straight without turning.
- Step 11: draw a normal line of 0.200 units. Then, continue straight without turning.
- Step 12: draw a triangle arc with a radius of 0.500 and sweeping 90.0 degrees. Then, continue straight without turning.
- Step 13: draw a triangle line of 0.400 units. Then, continue straight without turning.
- Step 14: draw a normal arc with a radius of 0.500 and sweeping 90.0 degrees. Then, continue straight without turning.

The figure is now complete.

Positive example.

To draw figure 1, follow these steps:

- Step 1: draw a normal line of 1.000 units. Then, continue straight without turning.
- Step 2: draw a normal line of 0.283 units. After that, turn left by 135.0 degrees.

Step 3: draw a normal line of 0.200 units. After that, turn left by 135.0 degrees.

Step 4: draw a normal line of 0.583 units. After that, turn right by 149.0 degrees.

Step 5: draw a normal line of 0.500 units. After that, turn left by 59.0 degrees.

Step 6: draw a normal line of 0.500 units. After that, turn left by 90.0 degrees.

Step 7: draw a square line of 0.200 units. Then, continue straight without turning.

Step 8: draw a triangle arc with a radius of 0.500 and sweeping 90.0 degrees. Then, continue straight without turning.

Step 9: draw a normal line of 0.400 units. Then, continue straight without turning.

Step 10: draw a square arc with a radius of 0.500 and sweeping 90.0 degrees. Then, continue straight without turning.

Step 11: draw a circle line of 0.200 units. Then, continue straight without turning.

Step 12: draw a normal arc with a radius of 0.500 and sweeping 90.0 degrees. Then, continue straight without turning.

Step 13: draw a triangle line of 0.400 units. Then, continue straight without turning.

Step 14: draw a triangle arc with a radius of 0.500 and sweeping 90.0 degrees. Then, continue straight without turning.

The figure is now complete.

A.5 Concept-Conditioned Prompt Fragment

When evaluating the Concept-Conditioned C–G setting, the same instance is annotated with a high-level rule:

Concept (system prompt fragment): “Here is the overall concept behind the positive samples: *unbalanced trapezoid right_triangle AND uneven band four arcs.*”

This example illustrates the three kinds of input used in our experiments for a single Bongard-LOGO problem: raw images (Figure A.1), procedural action programs (Table A.1), and natural-language action descriptions, optionally augmented with an explicit concept label.

B Model Details and Version Information

B.1 Proprietary Models

Table A.2: Proprietary Models Evaluated

Model	Developer	Release	Context Window	Multi-Image
Gemini-2.5-flash-preview	Google	May 2025	1M tokens	Yes
Gemini-3-flash-preview	Google	Dec. 2025	1M tokens	Yes

Access: Google AI Studio APIs. Requires authentication and API key.

B.2 Open-Source Models (via Ollama)

Infrastructure: All open-source models executed locally using Ollama framework on HPC cluster with heterogeneous NVIDIA GPUs (A100 40GB/80GB VRAM).

C Reproducibility and Implementation Details

C.1 Hardware Requirements

Minimum Configuration:

- GPU: NVIDIA RTX 3090 (24GB VRAM) or equivalent

- RAM: 64GB system memory
- Storage: 500GB for models and datasets

C.2 Software Stack

- Ollama v0.9.6 or later (model serving framework)
- Python 3.10+
- PyYAML for configuration parsing
- Requests library for HTTP communication
- Google API Python client for Gemini API access

C.3 Dataset Access

Bongard-LOGO Dataset: Available from original authors at: <https://github.com/NVlabs/Bongard-LOGO>

Dataset includes:

- 12,000 problem instances (3,600 Free-form, 4,000 Basic, 4,400 Abstract)
- Ground-truth labels and rule annotations
- Action program specifications for all images
- Human-designed shape library with 627 shapes and action programs

C.4 Code Availability

Complete experimental code, including:

- Data loading and preprocessing utilities
- Prompt generation functions (all variants in Appendix D)
- Model interaction wrappers (Ollama + Gemini API)
- Evaluation and result aggregation scripts
- Statistical analysis and visualization code

Code will be released upon paper acceptance at a public GitHub repository.

API usage for grounded calls. Grounded C–G calls differ from symbolic-only C–G only in that the query image is passed alongside the textual prompt. For Gemini models, we pass the prompt and a PIL image object; for Ollama-served VLMs, we add the image as a base64-encoded field in the system message. This setup ensures that *only* the query receives both symbolic and visual input, while positives and negatives remain purely symbolic, making Grounded C–G a minimal-anchoring ablation on top of the Componential–Grammatical interface.

C.5 Computational Costs

This work required substantial computational resources:

- 2000+ GPU-hours on HPC infrastructure

D Complete Prompts for All Experimental Conditions

This appendix provides the complete, unedited prompts used for each experimental condition. These prompts are essential for reproducibility and can be adapted for similar reasoning tasks.